NEXT ENGINEER · _SHEERAT_

Database Management System

BCS501

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 5 | **Credits:** 4

L-T-P: 3-1-0 | **Scheme:** AKTU B.Tech 3rd year, 2024-25 onwards

UNIT 1 Introduction and ER Modelling

UNIT 2 Relational Model, Algebra and SQL

UNIT 3 Database Design and Normalization

UNIT 4 Transaction Processing and Recovery

UNIT 5 Concurrency Control

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: comparison tables, rules and diagrams explained in words.
3. **Solved examples**: the problems that always appear (ER to tables, relational algebra, SQL, closure and keys, normalization, serializability).
4. **Code to remember**: SQL and PL/SQL you may be asked to write.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Units 1 and 2. Three-schema architecture, keys, ER to tables, relational algebra operators, and 10 SQL queries written by hand.
- **Afternoon (3 hours):** Unit 3. Attribute closure, candidate keys, minimal cover, then 1NF to BCNF with decomposition. This unit gives sure marks.
- **Evening (2 to 3 hours):** Units 4 and 5. ACID, schedules, conflict and view serializability with precedence graphs, recovery with logs and checkpoints, 2PL, timestamp ordering and multiple granularity. Then the "Last-hour checklist".
- **In the exam:** for normalization, always find the candidate keys first, state which normal form is violated and by which dependency, then decompose and check lossless join. For serializability, always draw the precedence graph.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction and ER Modelling

✍️ 2-mark definitions

- **Database:** an organized collection of related data stored so that it can be easily accessed, managed and updated.
- **DBMS:** software that defines, creates, stores, retrieves and manages a database and controls concurrent access to it, e.g. MySQL, Oracle, PostgreSQL.
- **Schema and instance:** the schema is the overall design (structure) of the database and changes rarely; an instance is the data stored at a particular moment.
- **Data model:** a collection of concepts for describing data, relationships and constraints, e.g. relational, ER, hierarchical, network, object-oriented.
- **Data independence:** the ability to change the schema at one level without changing the schema at the next higher level. Physical independence protects the conceptual schema from storage changes; logical independence protects external views from conceptual changes.
- **DDL and DML:** Data Definition Language defines structure (CREATE, ALTER, DROP); Data Manipulation Language accesses data (SELECT, INSERT, UPDATE, DELETE).
- **Entity and entity set:** an entity is a real-world object that can be distinguished from others; an entity set is a collection of similar entities.
- **Attribute types:** simple vs composite, single-valued vs multivalued, stored vs derived (e.g. age derived from date of birth), and key attributes.
- **Weak entity:** an entity set without a primary key of its own; it depends on an owner (identifying) entity and has a partial key (discriminator). Drawn with a double rectangle.
- **Super key:** any set of attributes that uniquely identifies a tuple.
- **Candidate key:** a minimal super key (no proper subset is a super key).
- **Primary key:** the candidate key chosen by the designer; it cannot be NULL. The others are alternate keys.
- **Foreign key:** an attribute in one relation that refers to the primary key of another relation.
- **Mapping cardinality:** the number of entities to which another entity can be associated: 1:1, 1:N, N:1, M:N.
- **Participation:** total (every entity takes part, double line) or partial (some entities take part, single line).

💡 Core concepts

File system vs DBMS

Point	File system	DBMS
Redundancy	high, same data in many files	controlled
Consistency	hard to maintain	enforced by constraints
Data access	write a new program for each query	query language (SQL)

Point	File system	DBMS
Concurrent access	anomalies	concurrency control
Security	file-level only	users, roles and views
Recovery	manual backups	automatic crash recovery
Integrity	coded in programs	declared as constraints

Three-schema architecture

- **External level (views):** what each user or application sees.
- **Conceptual level:** the logical structure of the whole database (tables, relationships, constraints).
- **Internal level:** how data is physically stored (files, indexes, blocks).

Mappings between levels give logical and physical data independence.

DBMS architecture (components): query processor (DDL interpreter, DML compiler, query evaluation engine), storage manager (authorization and integrity manager, transaction manager, file manager, buffer manager), and disk storage (data files, data dictionary, indexes). Users: naive users, application programmers, sophisticated users, and the DBA.

ER notation

Symbol	Meaning
Rectangle	entity set
Double rectangle	weak entity set
Diamond	relationship set
Double diamond	identifying relationship
Ellipse	attribute
Underlined ellipse	key attribute
Double ellipse	multivalued attribute
Dashed ellipse	derived attribute
Double line	total participation

Reduction of ER diagram to tables

1. Strong entity: one table with all simple attributes; composite attributes are split into components.
2. Weak entity: table with its attributes plus the owner's primary key; primary key = owner key + discriminator.
3. 1:1 relationship: add the primary key of one side as a foreign key in the other (prefer the total-participation side).
4. 1:N relationship: add the primary key of the "1" side as a foreign key in the "N" side.
5. M:N relationship: a separate table holding both primary keys (together the primary key) plus the relationship's own attributes.
6. Multivalued attribute: a separate table with the entity's key and the attribute.

Extended ER features

- **Specialization:** top-down; dividing an entity set into sub-groups (Employee → Engineer, Manager).
- **Generalization:** bottom-up; combining entity sets with common features into a higher-level set (Car, Truck → Vehicle). Shown with an ISA triangle.
- **Constraints:** disjoint vs overlapping, total vs partial.
- **Aggregation:** treating a relationship set together with its entities as a higher-level entity so it can take part in another relationship.

Relationships of higher degree: a ternary relationship (e.g. Supplier–Part–Project) cannot always be replaced by three binary relationships without losing information; it can be converted to binary by introducing an artificial entity set.

🧮 Solved examples

Example 1 (ER to tables). University: Student(roll, name, {phone}) M:N Course(cid, title) through Enrols(grade); Department(did, dname) 1:N Student.

Table	Columns	Key
Department	did, dname	did
Student	roll, name, did (FK)	roll
StudentPhone	roll (FK), phone	(roll, phone)
Course	cid, title	cid
Enrols	roll (FK), cid (FK), grade	(roll, cid)

Example 2 (weak entity). Loan(loan_no, amount) and Payment(pay_no, date, amt) where pay_no repeats per loan. Payment is weak; its table is Payment(loan_no, pay_no, date, amt) with primary key (loan_no, pay_no).

Key points to remember

- Every candidate key is a super key, but not every super key is a candidate key.
- Number of super keys for a relation with n attributes and one single-attribute candidate key is 2^(n−1).
- A weak entity always has total participation in its identifying relationship.
- Logical data independence is harder to achieve than physical data independence.

⭐ Important questions

1. Compare file systems and DBMS. Explain the three-schema architecture and data independence.
2. Explain the overall DBMS architecture with a neat diagram and types of users.
3. Explain keys (super, candidate, primary, alternate, foreign) with examples.
4. Draw an ER diagram for a given system (library, hospital, banking, university) and reduce it to tables.

5. Explain generalization, specialization and aggregation with diagrams.

NEXT ENGINEER
sheerat

UNIT 2 Relational Model, Algebra and SQL

✍️ 2-mark definitions

- **Relation:** a table of rows (tuples) and columns (attributes); the number of attributes is the degree and the number of tuples is the cardinality.
- **Domain:** the set of allowed atomic values for an attribute.
- **Integrity constraints:** rules that keep data correct. Domain constraint (value in domain), key constraint (unique key), entity integrity (primary key not NULL), referential integrity (a foreign key value must exist in the referenced table or be NULL).
- **Relational algebra:** a procedural query language with operators that take relations and return a relation.
- **Relational calculus:** a non-procedural query language that says what to retrieve, not how. Tuple relational calculus uses tuple variables: {t | P(t)}. Domain relational calculus uses domain variables: {⟨x1, ..., xn⟩ | P(x1, ..., xn)}.
- **View:** a virtual table defined by a query; it stores no data of its own.
- **Index:** a data structure that speeds up searching on a column at the cost of extra storage and slower updates.
- **Join:** combines rows of two tables on a condition. Natural join matches columns with the same name; outer joins also keep unmatched rows.
- **Subquery:** a query nested inside another query.
- **Cursor:** a pointer to the result set of a query, used in PL/SQL to process rows one at a time.
- **Trigger:** a stored PL/SQL block that runs automatically before or after an INSERT, UPDATE or DELETE on a table.
- **Stored procedure:** a named PL/SQL block stored in the database and called by name.

💡 Core concepts

Relational algebra operators

Operator	Symbol	Meaning
Select	σ condition (R)	rows that satisfy the condition
Project	π attributes (R)	chosen columns, duplicates removed
Union	R ∪ S	rows in either (union-compatible)
Set difference	R − S	rows in R but not in S
Cartesian product	R × S	every pair of rows
Rename	ρ new (R)	rename relation or attributes
Intersection	R ∩ S	rows in both; = R − (R − S)
Natural join	R ⋈ S	equal values on common attributes

Operator	Symbol	Meaning
Theta join	$R \bowtie_\theta S$	$\sigma_\theta(R \times S)$
Division	$R \div S$	values of R related to **all** rows of S
Left / right / full outer join	$\bowtie \; \bowtie \; \bowtie$	keep unmatched rows, pad with NULL

The basic (fundamental) operators are σ, π, $\cup$, $-$, $\times$ and ρ. All others can be written using them.

SQL command types

Type	Commands
DDL	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	SELECT, INSERT, UPDATE, DELETE
DCL	GRANT, REVOKE
TCL	COMMIT, ROLLBACK, SAVEPOINT

DELETE vs TRUNCATE vs DROP: DELETE removes chosen rows, can use WHERE, is logged and can be rolled back. TRUNCATE removes all rows quickly but keeps the table structure. DROP removes the table structure and data.

WHERE vs HAVING: WHERE filters rows before grouping; HAVING filters groups after GROUP BY and can use aggregate functions.

Order of a SELECT: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

Aggregate functions: COUNT, SUM, AVG, MIN, MAX. COUNT(*) counts all rows; COUNT(col) ignores NULLs.

Set operations: UNION (removes duplicates), UNION ALL (keeps them), INTERSECT, EXCEPT / MINUS.

🧮 Solved examples

Use these tables: Employee(eid, name, salary, dno), Department(dno, dname, city), Works(eid, pid, hours), Project(pid, pname).

Example 1 (relational algebra).

- Names of employees earning more than 50,000: $\pi_{name}(\sigma_{salary > 50000}(Employee))$.
- Names of employees in the "Sales" department: $\pi_{name}(Employee \bowtie \sigma_{dname = 'Sales'}(Department))$.
- Employees who work on every project: $\pi_{eid, pid}(Works) \div \pi_{pid}(Project)$.
- Employees who work on no project: $\pi_{eid}(Employee) - \pi_{eid}(Works)$.

Example 2 (division worked out). Works = {(1, P1), (1, P2), (2, P1), (3, P1), (3, P2)}, Project = {P1, P2}. Works ÷ Project = {1, 3}, because only employees 1 and 3 appear with both P1 and P2.

Example 3 (SQL).

```sql
-- Department-wise average salary, only departments with average above 40000
SELECT dno, AVG(salary) AS avg_sal
FROM Employee
GROUP BY dno
HAVING AVG(salary) > 40000;

-- Second highest salary
SELECT MAX(salary) FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);

-- Employees earning more than the average of their own department (correlated subquery)
SELECT e.name FROM Employee e
WHERE e.salary > (SELECT AVG(salary) FROM Employee WHERE dno = e.dno);

-- Every employee with department name, including those without a department
SELECT e.name, d.dname
FROM Employee e LEFT OUTER JOIN Department d ON e.dno = d.dno;

-- Employees who work on at least one project (EXISTS)
SELECT name FROM Employee e
WHERE EXISTS (SELECT * FROM Works w WHERE w.eid = e.eid);
```

Example 4 (tuple relational calculus). Names of employees earning more than 50,000: {t.name | t ∈ Employee ∧ t.salary > 50000}. Domain calculus: {⟨n⟩ | ∃ e, s, d (⟨e, n, s, d⟩ ∈ Employee ∧ s > 50000)}.

⌨️ Code to remember

```sql
CREATE TABLE Department (
  dno   INT PRIMARY KEY,
  dname VARCHAR(30) NOT NULL UNIQUE,
  city  VARCHAR(30) DEFAULT 'Lucknow'
);
CREATE TABLE Employee (
  eid    INT PRIMARY KEY,
  name   VARCHAR(40) NOT NULL,
  salary DECIMAL(10,2) CHECK (salary > 0),
  dno    INT REFERENCES Department(dno) ON DELETE SET NULL
);
CREATE VIEW HighPaid AS SELECT eid, name FROM Employee WHERE salary > 50000;
CREATE INDEX idx_emp_dno ON Employee(dno);
```

```
-- PL/SQL: explicit cursor
DECLARE
  CURSOR c IS SELECT name, salary FROM Employee WHERE dno = 10;
  v_name Employee.name%TYPE; v_sal Employee.salary%TYPE;
BEGIN
  OPEN c;
  LOOP
    FETCH c INTO v_name, v_sal;
    EXIT WHEN c%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_name || ' ' || v_sal);
  END LOOP;
  CLOSE c;
END;
/

-- Trigger: keep a log of salary changes
CREATE OR REPLACE TRIGGER trg_salary_log
AFTER UPDATE OF salary ON Employee
FOR EACH ROW
BEGIN
  INSERT INTO SalaryLog(eid, old_sal, new_sal, changed_on)
  VALUES (:OLD.eid, :OLD.salary, :NEW.salary, SYSDATE);
END;
/

-- Stored procedure: raise salary of a department
CREATE OR REPLACE PROCEDURE raise_salary(p_dno IN NUMBER, p_pct IN NUMBER) AS
BEGIN
  UPDATE Employee SET salary = salary * (1 + p_pct / 100) WHERE dno = p_dno;
  COMMIT;
END;
/
```

⭐ Important questions

1. Explain the integrity constraints of the relational model with examples.
2. Explain the relational algebra operators with examples; write queries for a given schema (including division).
3. Explain tuple and domain relational calculus with examples.
4. Write SQL queries for a given schema using joins, GROUP BY, HAVING, subqueries and set operations.
5. Explain views, indexes, cursors, triggers and procedures in PL/SQL with code.

UNIT 3 Database Design and Normalization

✍️ 2-mark definitions

- **Functional dependency (FD):** X → Y holds if any two tuples that agree on X also agree on Y.
- **Trivial FD:** X → Y where Y ⊆ X.
- **Armstrong's axioms:** reflexivity (if Y ⊆ X then X → Y), augmentation (if X → Y then XZ → YZ), transitivity (if X → Y and Y → Z then X → Z). Derived rules: union, decomposition, pseudo-transitivity.
- **Closure of attributes (X^+):** the set of all attributes functionally determined by X.
- **Canonical (minimal) cover:** an equivalent set of FDs with single attributes on the right, no extraneous attributes on the left, and no redundant FDs.
- **Prime attribute:** an attribute that is part of some candidate key.
- **Anomalies:** problems caused by redundancy: insertion, deletion and update anomalies.
- **Normalization:** decomposing relations to reduce redundancy and remove anomalies.
- **Lossless-join decomposition:** a decomposition of R into R1 and R2 where joining them gives back exactly R; it holds if R1 ∩ R2 → R1 or R1 ∩ R2 → R2.
- **Dependency preservation:** every FD of R can be checked using the FDs of the decomposed relations, without a join.
- **Multivalued dependency (MVD):** X →→ Y holds if, for each X value, the set of Y values is independent of the remaining attributes.
- **Join dependency (JD):** R satisfies JD(R1, ..., Rn) if R equals the natural join of its projections on R1 ... Rn.
- **Inclusion dependency:** values of attributes X in relation R must appear among the values of Y in relation S (foreign keys are an example).

💡 Core concepts

Normal forms

Normal form	Condition
1NF	every attribute holds atomic (single, indivisible) values
2NF	1NF and no partial dependency: no non-prime attribute depends on part of a candidate key
3NF	2NF and no transitive dependency: for every non-trivial X → A, X is a super key **or** A is prime
BCNF	for every non-trivial X → A, X is a super key
4NF	BCNF and for every non-trivial MVD X →→ Y, X is a super key
5NF (PJNF)	every join dependency is implied by the candidate keys

3NF vs BCNF

Point	3NF	BCNF
Rule	X super key or A prime	X super key
Strength	weaker	stronger (every BCNF relation is in 3NF)
Lossless decomposition	always possible	always possible
Dependency preservation	always possible	not always possible
Redundancy	small amount may remain	removed for FDs

Algorithm for attribute closure X^+. Start with result = X. Repeat: for each FD A → B, if A ⊆ result then add B to result. Stop when nothing changes. If X^+ contains all attributes, X is a super key.

Finding candidate keys quickly. Attributes that never appear on the right side of any FD must be in every key. Start with them, find the closure, and add attributes only if needed.

3NF synthesis algorithm: find a minimal cover; make one relation for each FD X → A (merge those with the same left side); if no relation contains a candidate key, add one that does; remove relations contained in others.

BCNF decomposition algorithm: while some relation R has an FD X → Y violating BCNF, replace R by R1 = $X^+ \cap R$ (or X ∪ Y) and R2 = R − (Y − X). This is always lossless.

🧮 Solved examples

Example 1 (closure and candidate keys). R(A, B, C, D, E), F = {A → BC, CD → E, B → D, E → A}.

- A^+: A → BC gives ABC; B → D gives ABCD; CD → E gives ABCDE. So A is a key.
- E^+: E → A, then as above, ABCDE. So E is a key.
- CD^+: CD → E, E → A, A → BC, giving ABCDE. CD is a key (C^+ = C and D^+ = D, so it is minimal).
- BC^+: B → D gives BCD; CD → E gives BCDE; E → A gives ABCDE. BC is a key.
- B^+ = BD, C^+ = C, D^+ = D; no other pair without A or E works except BC and CD.

Candidate keys: **A, E, BC, CD**. Prime attributes: all five, so this relation is in 3NF. B → D violates BCNF (B is not a super key).

Example 2 (minimal cover). F = {A → BC, B → C, A → B, AB → C}.

1. Split right sides: A → B, A → C, B → C, A → B, AB → C.
2. Remove the duplicate A → B.
3. In AB → C, A is extraneous because B → C already holds; it becomes B → C, a duplicate, so remove it.
4. A → C is redundant because A → B and B → C give it.

Minimal cover: **{A → B, B → C}**.

Example 3 (normalizing step by step). R(roll, course, name, dept, hod, marks) with roll → name, dept; dept → hod; (roll, course) → marks.

- Candidate key: (roll, course).
- 2NF violation: roll → name, dept (partial dependency on part of the key).
- 2NF: R1(roll, name, dept, hod), R2(roll, course, marks).

NEXT ENGINEER _sheerat_

- 3NF violation in R1: roll → dept → hod (transitive).
- 3NF: R11(roll, name, dept), R12(dept, hod), R2(roll, course, marks). All three are also in BCNF.

Example 4 (3NF but not BCNF). R(A, B, C), F = {AB → C, C → B}. Candidate keys: AB and AC. C → B: C is not a super key, so BCNF fails, but B is prime, so 3NF holds. BCNF decomposition: R1(C, B), R2(A, C). It is lossless (R1 ∩ R2 = C and C → B, so C is a key of R1), but AB → C is no longer preserved.

Example 5 (lossless check). R(A, B, C), F = {A → B}. Decomposition R1(A, B), R2(A, C): R1 ∩ R2 = A and A → B means A → R1, so it is lossless. Decomposition R1(A, B), R2(B, C): common attribute B determines neither side, so it is lossy.

Example 6 (4NF). Teaches(course, teacher, book) where a course's teachers and books are independent: course →→ teacher and course →→ book. Course is not a super key, so 4NF fails. Decompose into CT(course, teacher) and CB(course, book).

Key points to remember

- Any relation with only two attributes is in BCNF.
- If every candidate key is a single attribute, a relation in 1NF is automatically in 2NF.
- If all attributes are prime, the relation is automatically in 3NF.
- Lossless join is compulsory; dependency preservation is desirable.
- BCNF removes all FD-based redundancy; 4NF removes MVD-based redundancy; 5NF removes JD-based redundancy.

⭐ Important questions

1. What is a functional dependency? State Armstrong's axioms and prove the union and pseudo-transitivity rules.
2. Find the closure of attributes, candidate keys and minimal cover for a given set of FDs.
3. Explain 1NF, 2NF, 3NF and BCNF with examples; normalize a given relation up to BCNF.
4. Explain lossless-join and dependency-preserving decomposition with tests and examples.
5. Explain multivalued and join dependencies, 4NF and 5NF.

UNIT 4 Transaction Processing and Recovery

✍️ 2-mark definitions

- **Transaction:** a unit of program execution that accesses and possibly updates data items, using read(X) and write(X).
- **ACID properties:** Atomicity (all or nothing), Consistency (a correct state leads to a correct state), Isolation (concurrent transactions do not see each other's partial results), Durability (committed changes survive failures).
- **Transaction states:** active, partially committed, committed, failed, aborted (terminated after commit or abort).
- **Schedule:** a sequence of operations from a set of transactions that keeps the order of operations within each transaction.
- **Serial schedule:** transactions run one after another without interleaving.
- **Serializable schedule:** a concurrent schedule whose effect equals some serial schedule.
- **Conflicting operations:** two operations from different transactions on the same item where at least one is a write (R-W, W-R, W-W).
- **Conflict serializable:** a schedule that can be turned into a serial one by swapping non-conflicting adjacent operations; equivalently, its precedence graph has no cycle.
- **View serializable:** a schedule view-equivalent to a serial schedule (same initial reads, same reads-from, same final writes).
- **Recoverable schedule:** if Tj reads data written by Ti, then Ti commits before Tj commits.
- **Cascadeless schedule:** a transaction reads only data written by committed transactions, so one abort does not force others to abort.
- **Log:** a sequence of records on stable storage describing every update: ⟨Ti start⟩, ⟨Ti, X, old, new⟩, ⟨Ti commit⟩, ⟨Ti abort⟩.
- **Checkpoint:** a point at which the DBMS forces log records and modified buffers to disk, so recovery only needs to look at the log after it.
- **Distributed database:** a database stored at several sites connected by a network, appearing as one database to users.

💡 Core concepts

Conflict vs view serializability

Point	Conflict serializable	View serializable
Test	precedence graph acyclic	view equivalence to a serial schedule
Complexity	polynomial	NP-complete
Relation	every conflict-serializable schedule is view serializable	some view-serializable schedules are not conflict serializable (those with blind writes)

Precedence graph method. One node per transaction. Draw Ti → Tj if an operation of Ti conflicts with a later operation of Tj. If there is no cycle, the schedule is conflict serializable, and a topological order gives the equivalent serial order.

Hierarchy of schedules: strict ⊂ cascadeless ⊂ recoverable. A strict schedule does not let any transaction read or write an item until the last transaction that wrote it has committed or aborted.

Log-based recovery

Point	Deferred update	Immediate update
When DB is written	after commit only	before commit allowed
Log record	⟨Ti, X, new⟩ (no old value needed)	⟨Ti, X, old, new⟩
After a crash	redo committed transactions	undo uncommitted, redo committed
Undo needed	no	yes

Rules after a crash (immediate update): a transaction with both ⟨start⟩ and ⟨commit⟩ in the log is redone; a transaction with ⟨start⟩ but no ⟨commit⟩ or ⟨abort⟩ is undone. With a checkpoint, transactions committed before the checkpoint are ignored.

Write-ahead logging (WAL): the log record for an update must reach stable storage before the updated data is written to disk, and all log records of a transaction must be written before it commits.

Deadlock handling: prevention (order resources, or use timestamps: wait-die and wound-wait), detection (wait-for graph with a cycle), recovery (choose a victim and roll it back; avoid starvation).

- **Wait-die (non-preemptive):** if an older transaction requests a lock held by a younger one, it waits; if a younger one requests from an older one, it dies (rolls back).
- **Wound-wait (preemptive):** an older requester wounds (aborts) the younger holder; a younger requester waits.

Distributed databases.

- Storage: fragmentation (horizontal by rows, vertical by columns), replication (copies at several sites), or both.
- Transparency: users need not know where data is or how it is split.
- Distributed commit: two-phase commit (2PC). Phase 1: the coordinator sends "prepare" and each site votes ready or abort. Phase 2: if all voted ready the coordinator sends commit, otherwise abort.
- Distributed concurrency control: single lock manager, distributed lock manager, primary copy, majority protocol.
- Directory systems: specialized databases (e.g. LDAP) storing information about users and resources, optimized for lookups.

🧮 Solved examples

Example 1 (not conflict serializable). S: R1(A), R2(A), W1(A), W2(A).

- R1(A) before W2(A): T1 → T2.
- R2(A) before W1(A): T2 → T1.
- Cycle T1 → T2 → T1, so S is **not** conflict serializable.

Example 2 (conflict serializable). S: R1(A), W1(A), R2(A), W2(A), R1(B), W1(B), R2(B), W2(B). All conflicts go from T1 to T2, so the graph has only the edge T1 → T2. No cycle; equivalent serial order is **T1, T2**.

Example 3 (view serializable but not conflict serializable). S: R1(A), W2(A), W1(A), W3(A).

- Conflict graph: T1 → T2 (R1 before W2), T2 → T1 (W2 before W1), so there is a cycle; not conflict serializable.
- Compare with serial T1, T2, T3: T1 does the initial read of A in both; T3 does the final write in both; no transaction reads a value written by another. So S is view equivalent to T1 T2 T3 and is **view serializable**. W2 and W3 are blind writes.

Example 4 (recoverability). S: W1(A), R2(A), C2, C1. T2 reads A written by T1 and commits before T1. If T1 then aborts, T2 has committed with a wrong value, so S is **not recoverable**. Swapping to C1 before C2 makes it recoverable but not cascadeless; letting T2 read A only after C1 makes it cascadeless.

Example 5 (recovery with checkpoint). Log: ⟨T1 start⟩, ⟨T1, A, 100, 150⟩, ⟨T1 commit⟩, ⟨checkpoint⟩, ⟨T2 start⟩, ⟨T2, B, 200, 250⟩, ⟨T3 start⟩, ⟨T3, C, 50, 80⟩, ⟨T2 commit⟩, then crash.

- T1 committed before the checkpoint: no action.
- T2 committed after the checkpoint: **redo** (B = 250).
- T3 has no commit: **undo** (C = 50).

Key points to remember

- Atomicity and durability are ensured by the recovery system; isolation by concurrency control; consistency by the programmer and constraints.
- Precedence graph cycle → not conflict serializable (but may still be view serializable if blind writes exist).
- Recovery order: undo pass goes backward through the log; redo pass goes forward.
- 2PC is blocking: if the coordinator fails after "prepare", ready sites must wait.

⭐ Important questions

1. Explain the ACID properties and the transaction state diagram.
2. Define conflict and view serializability; test given schedules using a precedence graph.
3. Explain recoverable, cascadeless and strict schedules with examples.
4. Explain log-based recovery (deferred and immediate), checkpoints and write-ahead logging.
5. Explain deadlock handling in databases and distributed databases (storage, concurrency control, 2PC).

NEXT ENGINEER _sheerat_

UNIT 5 Concurrency Control

✍️ 2-mark definitions

- **Concurrency control:** techniques that ensure the isolation of concurrently running transactions and keep schedules serializable.
- **Lock:** a variable associated with a data item that controls access to it. Shared (S) lock for reading; exclusive (X) lock for reading and writing.
- **Two-phase locking (2PL):** a transaction acquires all its locks in a growing phase and releases them in a shrinking phase; once it releases a lock it cannot acquire any new one. It guarantees conflict serializability.
- **Strict 2PL:** 2PL where all exclusive locks are held until commit or abort; it gives strict (cascadeless) schedules.
- **Rigorous 2PL:** all locks (shared and exclusive) are held until commit.
- **Lock point:** the moment a transaction obtains its last lock; transactions are serialized in lock-point order.
- **Timestamp:** a unique number given to each transaction when it starts, TS(Ti). Each data item keeps W-timestamp(Q) and R-timestamp(Q).
- **Thomas write rule:** in timestamp ordering, an obsolete write (TS(Ti) < W-timestamp(Q)) is simply ignored instead of rolling back Ti.
- **Validation (optimistic) protocol:** a transaction runs in three phases: read, validation and write; conflicts are checked only at validation.
- **Multiple granularity:** locking at different levels of a hierarchy (database, file, page, record) using intention locks.
- **Multiversion scheme:** keeps several versions of each item so reads never wait; each read gets the right version.

💡 Core concepts

Lock compatibility

Held → / Requested ↓	S	X
S	Yes	No
X	No	No

Problems of concurrent execution without control: lost update (W-W), dirty read / temporary update (W-R by an uncommitted transaction), incorrect summary, and unrepeatable read (R-W).

Basic 2PL vs strict vs rigorous

Protocol	Serializable	Deadlock free	Cascadeless
Basic 2PL	yes	no	no

Protocol	Serializable	Deadlock free	Cascadeless
Strict 2PL	yes	no	yes
Rigorous 2PL	yes	no	yes
Conservative (static) 2PL	yes	yes (locks taken in advance)	not necessarily

Timestamp-ordering protocol.

- Ti issues read(Q): if TS(Ti) < W-timestamp(Q), reject and roll back Ti; otherwise read and set R-timestamp(Q) = max(R-timestamp(Q), TS(Ti)).
- Ti issues write(Q): if TS(Ti) < R-timestamp(Q), reject and roll back; if TS(Ti) < W-timestamp(Q), reject and roll back (or ignore it with Thomas write rule); otherwise write and set W-timestamp(Q) = TS(Ti).
- It is deadlock-free (no waiting), but may cause starvation and cascading rollbacks.

Validation-based protocol. Each transaction has Start(Ti), Validation(Ti) and Finish(Ti). TS(Ti) = Validation(Ti). Ti passes validation against every Tj with TS(Tj) < TS(Ti) if either Finish(Tj) < Start(Ti), or Tj finishes before Ti's validation and the items Tj wrote do not overlap the items Ti read. Good when conflicts are rare.

Multiple granularity locking. Lock modes: IS (intention shared), IX (intention exclusive), S, SIX (shared and intention exclusive), X. Rules: lock the root first; to lock a node in S or IS, the parent must be in IS or IX; to lock in X, SIX or IX, the parent must be in IX or SIX; lock top-down, unlock bottom-up; follow 2PL.

	IS	IX	S	SIX	X
IS	Yes	Yes	Yes	Yes	No
IX	Yes	Yes	No	No	No
S	Yes	No	Yes	No	No
SIX	Yes	No	No	No	No
X	No	No	No	No	No

Multiversion timestamp ordering. A write creates a new version Qk with W-timestamp and R-timestamp. A read by Ti returns the version with the largest W-timestamp ≤ TS(Ti); reads never fail. A write by Ti is rejected if some version has W-timestamp ≤ TS(Ti) < R-timestamp of that version. Multiversion 2PL gives read-only transactions a snapshot without locks.

Recovery with concurrent transactions. The checkpoint record lists the active transactions L. After a crash: build an undo-list and redo-list by scanning backward to the checkpoint (transactions with commit go to redo; those in L or started after, without commit, go to undo); undo backward, then redo forward.

Oracle case study (key points). Oracle uses multiversion read consistency with undo segments, so readers never block writers and writers never block readers. Row-level locking is the default; locks are never escalated. Isolation levels supported are READ COMMITTED (default) and SERIALIZABLE. Redo log files and archive logs give durability and point-in-time recovery; SYSTEM and data files are organized into tablespaces.

🧮 Solved examples

Example 1 (2PL schedule).

T1	T2
lock-X(A)	
read(A); A = A − 50; write(A)	
lock-X(B)	
unlock(A)	
	lock-X(A)
read(B); B = B + 50; write(B)	read(A)
unlock(B)	

T1 acquires both locks before releasing any, so it follows 2PL (lock point after lock-X(B)). It is not strict, because A is unlocked before commit; if T1 aborts, T2 must also roll back.

Example 2 (timestamp ordering). TS(T1) = 5, TS(T2) = 10, initially R-ts(Q) = W-ts(Q) = 0.

- T2: read(Q) succeeds; R-ts(Q) = 10.
- T1: write(Q): TS(T1) = 5 < R-ts(Q) = 10, so T1 is rolled back (a younger transaction already read the old value).

Example 3 (Thomas write rule). TS(T1) = 5, TS(T2) = 10. T2 writes Q (W-ts = 10), then T1 writes Q. Basic TO rolls T1 back; with Thomas write rule, T1's write is obsolete and simply ignored, and T1 continues.

Example 4 (deadlock in 2PL). T1: lock-X(A), then requests lock-X(B). T2: lock-X(B), then requests lock-X(A). Wait-for graph: T1 → T2 → T1, a cycle, so there is a deadlock. Roll back one of them (the victim).

Key points to remember

- 2PL guarantees conflict serializability but not freedom from deadlock or cascading rollback.
- Timestamp protocols never deadlock because transactions never wait.
- Intention locks tell higher levels what is happening below, so a coarse lock request can be checked without scanning every record.
- Multiversion schemes trade storage for read concurrency.

⭐ Important questions

1. Explain lock-based protocols: 2PL, strict 2PL, rigorous 2PL; prove 2PL ensures serializability.
2. Explain the timestamp-ordering protocol and Thomas write rule with examples.
3. Explain the validation-based protocol.
4. Explain multiple granularity locking with the compatibility matrix and a tree diagram.

5. Explain multiversion schemes and recovery with concurrent transactions; give a case study of Oracle.

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ File system vs DBMS; three-schema architecture; data independence.
- ☐ Keys: super, candidate, primary, alternate, foreign; ER symbols; ER to tables rules.
- ☐ Specialization, generalization, aggregation.
- ☐ Relational algebra operators including division and outer joins.
- ☐ SQL: GROUP BY with HAVING, second highest salary, correlated subquery, joins.
- ☐ Cursor, trigger and procedure syntax.
- ☐ Attribute closure; candidate keys; minimal cover.
- ☐ 1NF to BCNF rules; 3NF vs BCNF; lossless test (R1 ∩ R2 → R1 or R2).
- ☐ MVD and 4NF, JD and 5NF.
- ☐ ACID; transaction states; precedence graph; view serializability with blind writes.
- ☐ Recoverable vs cascadeless vs strict; log recovery with checkpoint (redo committed, undo the rest).
- ☐ 2PL variants; timestamp rules; Thomas write rule; validation; multiple granularity matrix; 2PC.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Database System Concepts	Abraham Silberschatz, Henry F. Korth, S. Sudarshan	McGraw Hill	WorldCat
2	An Introduction to Database Systems	C. J. Date	Addison Wesley	WorldCat
3	Fundamentals of Database Systems	Ramez Elmasri, Shamkant B. Navathe	Addison Wesley	WorldCat
4	Database: Principles, Programming, and Performance	Patrick O'Neil, Elizabeth O'Neil	Elsevier	WorldCat
5	Database Management Systems	Raghu Ramakrishnan, Johannes Gehrke	McGraw Hill	WorldCat
6	Database Management Systems	Alexis Leon, Mathews Leon	Vikas Publishing House	WorldCat
7	An Introduction to Database Systems	Bipin C. Desai	Galgotia Publications	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
8	Database Management System	A. K. Majumdar, P. Bhattacharyya	Tata McGraw-Hill	WorldCat

Syllabus source: AKTU B.Tech 3rd year syllabus, 2024-25 (copy hosted by an affiliated college, PDF)

NEXT ENGINEER
sheerat